Taming Revocable Spot VM Instances with DIFFUSED

Zhao Zhang
Georgetown University
Washington, D.C., USA

Zeying Zhu
University of Maryland
College Park, MD, USA

Micah Sherr
Georgetown University
Washington, D.C., USA

Benjamin E. Ujcich
Georgetown University
Washington, D.C., USA

Wenchao Zhou
Georgetown University
Washington, D.C., USA

***Abstract*—Modern clouds offer *spot virtual machine (VM) instances* that provide cost savings but leave unsolved technical debt, which forces users to handle unpredictable revocations. We introduce DIFFUSED, a Linux runtime shim that enables *process diffusion*—the ability to seamlessly roam the *execution* of programs across instances. DIFFUSED decouples revocation handling, allowing the use of spot instances as cheaper, diskless, and replaceable computation nodes. We evaluate DIFFUSED and show that it provides similar (or even improved) performance to on-demand instances while maintaining up to a 52% spot discount.**

***Index Terms*—Cloud computing, virtual machines, spot instances, process migration, process diffusion, resource management, distributed systems**

I. INTRODUCTION

Cloud providers pack unused virtual machine (VM) capacity into *spot instances* to increase resource utilization and service monetization [1, 2]. Spot instances are less expensive than on-demand instances[1] (e.g., 60–70% discount for AWS EC2 [3]), but impose the trade-off that cloud providers may *revoke* (terminate and reclaim) allocated spot instances at any time, causing a two-fold technical debt. First, spot instances have to assume cloud tenants can stop their (or their users') tasks and save program state safely within a short time window (e.g., 120s on AWS EC2 [4]), which is not guaranteed. Second, converting traditional programs, which are designed for a stable environment (e.g., an on-demand instance), into revocation-friendly serverless programs requires extensive work. Unfortunately, designing workloads that work within these constraints can be difficult and has limited spot instance use to date [5].

A general approach to handle *spot instance revocation* is to migrate affected tasks off of revoked instances before they are reclaimed. However, VM-level migration [6–8] requires relocating both OS memory state and storage, which can often take longer than the time allowed by the revocation window [9]. Process-level migration [10, 11] is more efficient and offers finer-grained control, but still requires substantial manual effort to ensure that storage and other components of the process' environment remain consistent between checkpointing and restoration across different instances. Alternatively, other approaches tackle revocation by switching between spot and on-demand instances only for "recoverable" tasks that are safe to move across instances [12] or by requiring non-generalized domain-specific solutions [13–16]. In summary, securing both memory and storage states can be hard to fit into the short time window, and guaranteeing a consistent environment can be tedious.

In this paper, we argue for a hybrid approach in which tenants can leverage a powerful spot instance and a stable (and potentially low-cost) on-demand instance. The key insight of our approach is to decouple a running process into (i) a *pure computational part* that can be freely moved across (spot) instances, and (ii) an *environmental part* that consists of interactions with the environment (e.g., storage I/O) which are handled by a (potentially low-cost) on-demand instance.

We introduce *process diffusion*, a migration-like design that starts a process from an on-demand instance and then *roams* its computational part across (spot) instances. Opened resources (e.g., file descriptors) are preserved on the on-demand VM and are accessed from spot instances via remote procedure calls (RPCs). When revocation occurs, the computational part running on a spot instance can be quickly *recalled* to the on-demand instance within the revocation window. The process runs there until a new spot instance becomes available, at which point it can be roamed again.

We present an implementation of process diffusion called DIFFUSED. DIFFUSED operates at the process granularity and uses a lightweight shim, DHOST, that lives within the target program's process space. DHOST enables transparently moving the computational part of a process to a spot instance with access to the original instance's resources. DIFFUSED provides an API for accepting roam commands that can be integrated with management/orchestration systems. DIFFUSED also provides a global monitor, DMON, that oversees all instances and makes roaming decisions according to user-specified policies.

DIFFUSED eliminates the need to handle underlying stor-

[1]In this paper, we use "regular" or "on-demand" instances to represent non-revocable instances that clouds cannot proactively terminate. Dedicated or reserved instances also have similar non-revocable properties.

age during revocation and provides a consistent environment across instances. DIFFUSED is compatible with modern Linux kernels and many POSIX single- or multi-threaded programs, without modifying the OS and with minimal modifications to the application. We evaluate DIFFUSED on AWS EC2 as a representative cloud, though DIFFUSED provides extensible APIs for other providers. We demonstrate DIFFUSED's efficacy through use cases with various programs and workloads. We find that DIFFUSED can recall execution within seconds upon revocation for CPU-bound programs.

In summary, this paper makes the following contributions:

1) Process diffusion, a lightweight migration-like design that enables efficient and transparent roaming of running processes' execution flows across instances. Process diffusion allows tenants to take advantage of spot instances' performance and discounts without manual revocation handling.
2) The DIFFUSED design and implementation, encompassing DHOST that combines process diffusion with automatic revocation handling, and DMON, an extensible manager that orchestrates tasks and instances.
3) An extensive performance evaluation of DIFFUSED using a representative sampling of applications. We find that DIFFUSED provides up to 52% cost savings compared to relying solely on on-demand instances.

We maintain a project page for DIFFUSED at https://guseclab.github.io/diffused/.

II. MOTIVATION AND RELATED WORK

Among various resource-optimized instances (see Table I), compute-optimized instances offering greater memory and more vCPUs are often expensive. Their spot counterparts are highly-discounted, but can be revoked unexpectedly [17].

Prior work proposes a number of techniques for managing the possibility of revocation. For example, SNAPE [18] performs learning-based prediction of instance revocation, and Wu et al. propose a model to switch between spot and on-demand instances [12] to increase the availability of spot instances. However, neither approach provides concrete solutions for revocation handling: the former cannot guarantee complete availability, and the latter requires running "recoverable" tasks, leaving VM users to handle revocations. Additionally, there are migration-based solutions for moving nested VMs between spot instances upon revocation [6, 7], but they can still exceed revocation's time window when incremental checkpoints are not ready or the VM is too large.

Other designs either (i) use spot instances in a domain-specific manner, such as ML inference [19] with internal checkpointing or memory caches [14, 15] with fault-tolerant designs, or (ii) work only for specific types of workloads such as batched workloads [8], MPI applications [20], web applications [16], and mostly-idle applications [21]. As a result, without a general-purpose solution for revocation handling, spot instance tenants are required to enumerate all affected programs and find individual solutions for revocation.

TABLE I: Example EC2 pricing [3], where `i4i`, `r6i`, and `c6i` are storage-, memory-, and compute-optimized instances, respectively. NVMe storage is attached to `i4i` instances. We use `.l` and `.xl` as shorthands of `.large` and `.xlarge`.

Type	vCPU / Mem. (GiB)	Net. (GBit)	On-demand Price ($/hr)	Spot Price ($/hr)
`i4i.l`	2 / 16	≤ 10	0.172	.0447 (⇓ 74%)
`r6i.l`	2 / 16	≤ 12.5	0.126	.04788 (⇓ 62%)
`r6i.xl`	4 / 32	≤ 12.5	0.252	.09828 (⇓ 61%)
`r6i.2xl`	8 / 64	≤ 12.5	0.504	.21168 (⇓ 58%)
`c6i.l`	2 / 4	≤ 12.5	0.085	.0391 (⇓ 54%)
`c6i.xl`	4 / 8	≤ 12.5	0.170	.0714 (⇓ 58%)
`c6i.2xl`	8 / 16	≤ 12.5	0.340	.1292 (⇓ 62%)
`c6i.4xl`	16 / 32	≤ 12.5	0.680	.2448 (⇓ 64%)
`c6i.8xl`	32 / 64	12.5	1.360	.4488 (⇓ 67%)
`c6in.l`	2 / 4	≤ 25	0.1134	.0385 (⇓ 66%)
`c6in.xl`	4 / 8	≤ 30	0.2268	.0793 (⇓ 65%)
`c6in.2xl`	8 / 16	≤ 40	0.4536	.1905 (⇓ 58%)
`c6in.4xl`	16 / 32	≤ 50	0.9072	.3447 (⇓ 62%)
`c6in.8xl`	32 / 64	50	1.8144	.7257 (⇓ 60%)

Key insight: Tenants must currently provide domain-specific solutions to handle revocation using spot instances.

A. Observations and challenges

We highlight several observations and corresponding challenges concerning spot instances and revocation handling.

Correct resources in the wrong shape. In general, the idle resources behind spot instances are what the cloud wants to sell and what a tenant (customer) wants to buy. However, packing such resources into revocable VMs poses challenges and places the burden of promptly handling revocation on tenants.

Large states vs. short time window. Spot instances can be loaded with arbitrary programs and large amounts of data. However, tenants must stop using instances within a concrete time limit after the revocation notification (e.g., two minutes) and save all memory and storage states to a safe location before revocation, which is challenging.

Continuous experience vs. interruptible resources. Tenants usually expect a continuous experience without noticeable freezes or interruptions, whereas spot instances can be revoked and terminated at any time. Maintaining continuous execution of affected tasks is also challenging when spot instance termination must be handled concurrently.

Key insight: Revocation handling must ensure timely completion, ensure a seamless experience, and enable the tenant to take advantage of lower-cost spot instances.

B. A hybrid between migration and offloading

We propose *process diffusion* as a design that sits between process migration and program offloading approaches:

Process migration. Process migration follows a checkpoint/restore (C/R) workflow in which program state is first collected, marshaled, and then transported to another node, where it is then unmarshalled before resuming execution.

Live migration techniques include in-kernel solutions that may require customized kernels or kernel modules (e.g., Linux-CR [22]), and userspace solutions such as CRIU [10] and DMTCP [11] with finer granularity.

Migration approaches focus on migrating complete program instances *and their environment* (e.g., program-related storage). Process migration requires that all of the processes' dependencies (e.g., dynamically linked libraries) and mounted file systems are present and stay consistent on the new host. Migrating the environment can be expensive (e.g., underlying storage must be migrated), preventing the use of spot instances *disklessly*, and can run into potential corner cases (e.g., handling packetized pipes) [23, 24].

Key insight: Process migration adds complexity by requiring storage management during revocation, which prevents spot instances as purely diskless compute nodes.

Program offloading. Program offloading approaches allow a program to offload a partial workload (e.g., functions or threads) to other machines and scale out dynamically in response to a load spike, and achieve better performance by carefully pairing workloads and resources. Serverless programming [25–27] or similar approaches [28] enable programmers to decouple software into a set of functions or other small units (e.g., proclets [28]). However, these approaches have their own limitations (e.g., AWS Lambda functions are terminated after 15 minutes) and, more importantly, require programs to be (re)written as small functional units. Other offloading designs try to avoid requiring source code modifications by customizing the OS kernel and compilers [29, 30] or by depending on specific language features provided by managed languages such as Java [31].

Key insight: Compared with offloading approaches, we want to provide better compatibility with unmodified mainline kernels, compilers, and applications.

III. DIFFUSED OVERVIEW

We present DIFFUSED, our process diffusion realization. Fig. 1 shows a representative workflow.

A. Deployment scenarios

DIFFUSED works with a mix of one on-demand VM instance and one or more spot instances. In this paper, we focus our description on AWS EC2 instances [1], though DIFFUSED also works with other cloud providers, including GCP [32]. We assume that on-demand VMs are non-revocable with high availability [33], and revocation notifications of spot instances are received two minutes before termination [17].

This paper primarily targets spot instances with $\leq$ 32 vCPUs and 64 GiB of memory. Hadary et al. [34] found that more than 95% of Azure VM users provision instances with $\leq$ 32 vCPUs, indicating this configuration covers most cloud tenants' needs. We target VMs with 64 GiB or less of memory since such instances are generally safer for cloud networks to transfer memory states within the 120s revocation window. While

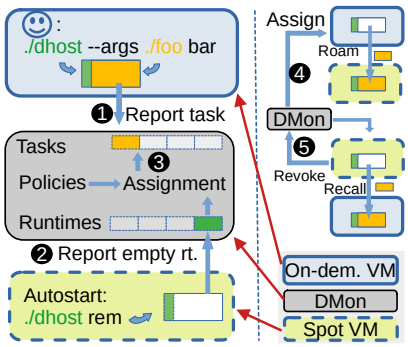


Fig. 1: The DIFFUSED workflow. ❶ A program `foo` starts with DHOST (shared process) from an on-demand VM and is reported to DMON as a task. ❷ An empty DHOST starts from a spot VM and is reported to DMON as a new empty runtime. ❸ DMON monitors tasks and runtimes, and then assigns an empty runtime for task (`foo`) based on pre-defined policies. ❹ DMON notifies the DHOST running task `foo` to roam (i.e., move) to the spot instance. ❺ DMON receives a spot instance termination notification and recalls the roamed program.

DIFFUSED is not limited to this configuration, we believe this choice covers commonly used cloud setups.

DIFFUSED allows mixing multiple types of on-demand and spot instances (see Table I), enabling the user to keep an always-on moderate on-demand instance (e.g., `c6i.large`) and request more powerful spot instances (e.g., `c6i.8xlarge`) only when needed. DIFFUSED assumes a fast, always-available network between on-demand and spot instances. All spot and on-demand instances should stay within the same availability zone (AZ) to benefit from free data transfer costs [35].

The cost of a DIFFUSED deployment with one on-demand instance and N spot instances can be calculated as:

$$C_{\text{total}} = P_O \cdot T_O + \sum_{i=0}^{i<N} P_{Si} \cdot T_{Si} \quad (1)$$

where P_O and P_{Si} are the on-demand and spot instance prices, and T_O and T_{Si} are the respective instance utilization times.

B. A typical DIFFUSED workflow

DIFFUSED includes two major components: the DHOST runtime and the DMON monitor (see Figure 1). We use the term *original* to describe the DHOST process inside the on-demand VM and *remote* to refer to those spawned from spot VMs.

Invocation. To use DIFFUSED, a user invokes a program on the original host by prepending the `dhost` command, e.g.:

```
dhost --args ffmpeg -i video.mpg video.avi
```

DHOST shares the same process space with the target program (e.g., `FFmpeg`). DHOST loads the dynamic loader, the target program, and all dependent libraries into a managed address space and transfers control to the target program. DHOST reports to DMON as a *task runtime* (Figure 1, ❶).

Spot VMs use a custom DHOST-equipped system image (e.g., an Amazon Machine Image (AMI)). The remote DHOST reports to DMON as an *empty runtime* (Figure 1, ❷). Remote DHOSTs can be manually started or started inside autoscaling fleets or groups [36].

Monitoring. DMON oversees (i) task and DHOST runtimes and (ii) spot instance revocations, and assigns empty runtimes for tasks. DMON supports customizable policies that describe when to perform roams and recalls (Figure 1, ❸). For example, we define an "assign-when-available" policy that assigns empty runtimes to tasks whenever possible.

Process diffusion. When a DHOST receives the *roam* command, it pauses the target program and copies the CPU context, memory layout, and thread information to the remote host. The remote DHOST resumes the target program's execution (Figure 1, ❹). When a process roams, CPU and physical memory usage on the original VM is reduced.

Handling revocation. A roamed process can be recalled back to the original DHOST by DMON upon revocation (Figure 1, ❺). This design turns the problem of creating a bespoke domain-specific spot instance revocation mechanism into simply recalling a roamed process. After a recall, DMON can roam the process to the next available spot instance.

C. Selecting instance types

DIFFUSED allows cloud tenants to decide between (i) cost savings and (ii) fixed costs and improved cloud performance. Although DIFFUSED does not impose any fixed strategy, a reasonable initial-step is for the tenant to first determine its resource needs (as it would ordinarily without DIFFUSED). For example, this might be a `c6i.4xlarge` instance with 16 vCPU and 32 GiB memory that costs $0.68/hr (see Table I). Given this starting point, the tenant can either (i) construct a DIFFUSED deployment with the same spot instance (`c6i.4xlarge` at $0.2448/hr) and a smaller on-demand instance (e.g., `c6i.large` with 2 vCPU and 4 GiB memory at $0.085/hr) and thus save $0.3502/hr (48.5%), or (ii) use a more powerful spot instance (e.g., `c6i.8xlarge`, 32 vCPU, 64 GiB memory at $0.4488/hr) that fits within its fixed budget.

To avoid swap thrashing when recalling memory-intensive programs, the on-demand instance should have at least as much memory as the spot instance, where a cheap memory-optimized (`r6i`) on-demand instance is recommended.

IV. PROCESS DIFFUSION WITH DIFFUSED

DIFFUSED consists of (i) the DIFFUSED container shim (DHOST) to manage the lifecycle of the target program across instances and (ii) the DIFFUSED monitor (DMON) that monitors DHOST runtimes and orchestrates roam and recall operations based on policies and revocation notifications.

A. DHOST: A shared process design

DHOST is designed for general-purpose programs and requires OS-level functionalities to diffuse a running process, similar to Library Operating Systems (LibOSes) or unikernels [37–39].

Inspired by DMTCP [11] and Gramine [40], DHOST is compiled into a static Position Independent Executable (PIE) and is loaded in isolation from the target program, ensuring zero interference between its internal functions and the target program. DHOST employs system call interposition to provide consistent behavior (see § IV-D). DHOST intercepts system calls via a modified musl-libc[2], and supports a number of unmodified binaries from Alpine Linux's apk repo [42] (which use musl-libc by default).

Such a design brings several benefits. No kernel modifications or nested virtualizations [43] are required. DHOST's shared-process design eliminates (1) the IPC overhead if it were to operate as a separate process and (2) user-kernel context switches if it were a kernel module. By staying in its isolated space, DHOST can maintain its own threads, sockets, and file descriptors to manage the target program.

B. Achieving process diffusion

As detailed below, DHOST pauses the running program upon diffusion commands. It then transfers states to the remote DHOST and restores the execution remotely.

Stage 1: Pause the target program: Like DMTCP [11], DHOST registers a `SIGUSR2` signal handler at startup. Upon receiving the roam command, DHOST enumerates all threads of the target program and sends `SIGUSR2` to each via *tgkill()*. This invokes DHOST's signal handler in its isolated memory regions and avoids interfering with the program's states.

Stage 2: Transfer execution flow using ucontext: The ucontext data structure, which contains the CPU register states of a thread, is marshaled over DIFFUSED's RPC channel to the destination DHOST runtime, as shown in Figure 2(a). DHOST sends only the minimal information required to diffuse the execution, including (1) the ucontexts of all target threads and (2) either allocated memory content, or, if user-fault file descriptors (UFFD) is enabled (details in § IV-C), solely the target program's memory layout as discerned via /proc/maps. The ucontext is valid across different instances as long as the CPU architecture and the virtual memory layout are compatible. DIFFUSED assumes the former (as a design requirement) and ensures the latter (see § IV-C).

Stage 3: Resume on the remote host: The remote DHOST runtime then spawns threads, restores the marshaled ucontexts, and resumes the target program's execution.

Backward execution recall: Once roamed, a program may be recalled to its original instance (e.g., upon revocation). The recall procedure is similar to the roaming procedure.

A benefit of DIFFUSED's design is that the remote DHOST runtime does not need the binary program nor dependency libraries a priori. A cold-started spot instance only needs to load the 800 KB DHOST from a snapshot. The target

[2]In our DIFFUSED implementation, a program (1) must be linkable to musl-libc [41], an alternative C standard library to glibc, and (2) should not trigger system calls via ASM *syscall* instructions (via *syscall(2)* is fine). Musl-libc simplifies our implementation but is not critical to DIFFUSED's design.

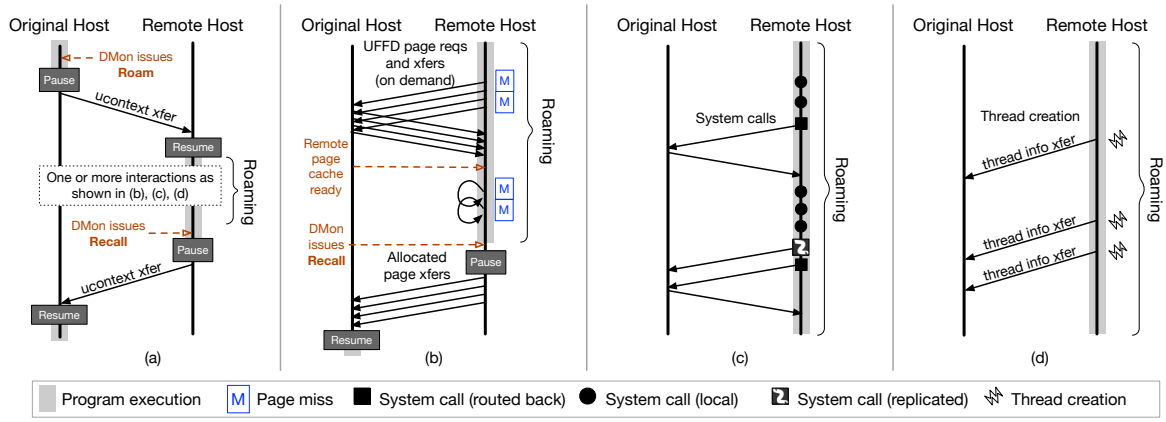


Fig. 2: Communication between DHOST runtimes. *(a)* Thread contexts (e.g., program counters and stack pointers) are communicated during process diffusion (see § IV-B). *(b)* When executing on the remote runtime, pages are fetched on demand. Repeated accesses for the same page do *not* require additional fetches (see § IV-C). *(c)* System calls are handled either locally, proxied back to the original host, or executed on both hosts (see § IV-D). *(d)* When threads are created when the target program is roaming, thread information is relayed to the original host (see § IV-F).

program's binary and libraries are transferred from the on-demand instance (§ IV-C), enabling diskless spot instances and avoiding the initialization overhead of cold storage (e.g., Amazon's Elastic Block Storage, EBS [44]).

C. Ensuring cross-instance memory consistency

Distributed shared memory (DSM) provides cross-instance memory consistency [45, 46], at the cost of higher latency [47] compared to local memory accesses. Even with state-of-the-art RDMA or Compute Express Link (CXL) hardware [48, 49], remote memory access is significantly slower than local accesses [49, 50]. DHOST employs several strategies for consistent and fast memory access:

Ground the target program's memory. DHOST controls the allocation of the program's address space by intercepting mmap/munmap calls and assigning consistent addresses across instances with the MAP_FIXED option [51].

Diffuse with memory consistency. By default, DHOST sends all allocated memory upon roam and recall to guarantee consistency. DHOST also supports a userfaultfd-based [52] lazy memory migration approach (UFFD) similar to an in-place page-server [53]. With UFFD, the original side DHOST sends only the program's memory *layout* and the remote DHOST restores the layout (*not* contents) before resuming (via mprotect and madvise(MADV_DONTNEED)).

When the program resumes on the remote host, it triggers page fault events that are sent to the UFFD handler thread (as part of the DHOST). As shown in Figure 2(b), the UFFD handler thread then fetches the accessed page from the original runtime, installs it in memory, and wakes up the paused thread(s). The UFFD handler no longer watches an installed page. Fetching pages via RPC imposes network latency on the critical path of program execution. To avoid this, after roam, the original DHOST sends all program memory pages to a *remote page cache* maintained by the remote DHOST.

During recall, all allocated memory is sent back to the original instance (before the revocation window elapses). The target program resumes *after* receiving all memory content.

Relaxed munmap when the program roams. When a program is roamed to a remote host, the original host keeps only a minimal amount of its memory active—mainly to handle forwarded system calls. To ensure that execution can continue seamlessly across hosts, DHOST enforces a consistent virtual memory layout with synchronized read/write permissions. To avoid the overhead of repeatedly calling mmap and munmap, the remote host maintains a list of previously unmapped memory regions and reuses them when possible, instead of actually unmapping them on the original host. If no reusable region is available, the system allocates a large memory block (for example, 128 MB) and uses it to replenish the list of free regions.

D. Handling system calls

Unlike early Single System Image (SSI) designs (e.g., Sprite [54] and MOSIX [55]) that forward *all* system calls back to a "home" machine (and also do not support multi-threaded programs [56]), DIFFUSED prefers local system call execution (at the spot instance) including futex [57], which is used by many lock implementations. However, storage and some memory system calls must be forwarded to the original host.

DIFFUSED intercepts system calls with a patched musl-libc that wraps system calls with a *pre-handler* and a *post-handler*. The pre-handler handles bookkeeping, argument modification, and syscall-forwarding jobs. If it intercepts a system call that must be sent back, it also sends the necessary memory states (following system call definitions) and waits for the updated memory states, return value, and errno. The post-handler executes after the system call finishes and handles bookkeeping tasks, such as handling the return value. Based on our experience, sending a minimal system call would incur

a $150\mu s$ latency per call, so our design prefers *in-place* system call execution when roaming. Upon a roam/recall, DHOST waits for unblocking system calls to finish before sending signals to corresponding threads.

Additionally, DHOST modifies the results of some system calls for other purposes. For example, many programs use *sched_getaffinity()* to determine the number of available cores; DHOST modifies the result with the available cores of the spot instance so that the target program can leverage the spot instance's greater CPU capacity.

E. Handling networked programs

To prevent forwarding socket-related system calls, DHOST monitors all listening sockets, epollfds, and epoll events in order to restore them after a move. Upon roam/recall, existing data connections are interrupted at the OS level, and the program runs for an additional 5–10 seconds, assuming it can gracefully close data sockets before pausing execution. Once moved, the program accepts reconnecting clients. Our assumption is that the Internet works in a best-effort manner and networked programs should handle disrupted connections correctly.[3] We envision a future integration with a kernel-bypassing network layer (e.g., AF_XDP-based iip [59]) to allow roam/recall with existing connections [60].

F. Enabling multithreading

DIFFUSED supports POSIX multithreaded programs. During thread creation, DHOST records threads' TLS (thread local storage) information. When a roamed program creates a new thread, DHOST creates an identical sleep thread on the original side (Figure 2(d)). This new thread begins execution when/if the target program is recalled back. Thread termination is noted to ensure consistency between the two DHOSTs. To bookkeep paired threads, DHOST uses a virtual-thread ID (TID). Overall, coherency between threads is achieved via the combination of DIFFUSED's consistent memory model (see § IV-C), the local execution of futex (see § IV-D), and DIFFUSED's requirement that all threads are moved together.

G. Communicating between DHOSTS

DIFFUSED uses a memory channel for memory synchronization and an RPC channel for all other communication. Both are built with standard TCP sockets in favor of lower-latency RPC frameworks to maximize compatibility. Nagle's algorithm [61] is deactivated for shorter latency, and DIFFUSED aims to *minimize the need* to communicate (e.g., by serving system calls as locally as possible).

H. DMON: the DIFFUSED monitor

DMON is an extensible monitor that manages roams and recalls of DHOST runtimes, and exposes a simple API that consists of *roam()* and *recall()* calls. A tenant can extend roaming and recalling policies as needed. For instance, a simple policy might perform process diffusion as long as there are empty DHOST runtimes on spot instances, as we demonstrate in § V. We consider policy design to be orthogonal to our main contribution of process diffusion, but we provide the necessary primitives for more complex policies (e.g., SNAPE [18] or Uniform Progress [12]) to be implemented.

I. Implementation

We implemented DIFFUSED for x86_64 Linux with 8.9k LoC for DHOST and 389 LoC for DMON. We patched ∼100 lines in musl-libc. We disabled ASLR for ease of development; supporting it requires sharing the random offset between DHOST runtimes and is left as a future enhancement.

V. EVALUATION

Our evaluation consists of (i) benchmarks with bare-metal machines that measure the overhead of DIFFUSED; (ii) performance benchmarks using AWS EC2 spot instances; and (iii) studies of alternative revocation handling approaches. We evaluate the following benchmarks:

1) **SimpleKV** is a program we wrote to micro-benchmark pathologically controlled behaviors where all clients are local threads and no system calls or memory allocations happen during the measurement.
2) **BusyBox (dd)** [62] is Alpine Linux's builtin toolkit. We benchmark EC2 EBS and NVMe disk-IO throughput using its `dd` function by duplicating a 1.3 GB file.
3) **FFmpeg** [63] is a multi-threaded CPU-intensive media converter. We evaluate the performance of converting a 1.3 GB .mp4 video into .avi format.
4) **Lbzip2** [64] is a multi-threaded CPU-intensive compressor. We make small modifications to remove its use of SIGUSR2. We benchmark the compression time of the Linux kernel 6.5 source code from a 23.2 GB .tar file into a 6.3 GB .bz2 file.
5) **Llama.cpp** [65] is a C++ backend that allows CPU-only local deployment of small LLMs (Large Language Models). We use its builtin `llama-bench` to evaluate the prompt process (pp) and text generation (tg) performance of the Meta-Llama-3.1-8B-Q_8 [66] and DeepSeek-R1-Distilled-Qwen-14B-Q6_K [67] models.
6) **Memcached** [68] is a memory object caching system. We use `memtier_benchmark` [69] for benchmarks (200 connections, 60s, 640 bytes value, 9:1 read/write ratio).
7) **PARSEC 3.0** [70] is a benchmarking suite for measuring scalability. We select programs that run $\geq 20s$ with 32 worker threads at native-scale on a ChameleonCloud [71] (compute_icelake_r650) hosted machine.
8) **Python Tornado** [72] is a single-threaded web framework. We host a 23 KB static webpage with 20 HMAC header cookies to simulate computation workload and benchmark with `wrk` [73] (200 connections for 60s).

We use `stress-ng` [74] to simulate contention and AWS Fault Injection Service (FIS) [75] to trigger spot instance revocations. All individual trials are repeated ten times and reported with mean (and standard deviation) unless otherwise

[3] A program can always listen to the IP address `0.0.0.0` to avoid handling different interface IPs across instances. Additionally, AWS provides elastic network interfaces (ENI) [58] that enable the detachment/re-attachment of secondary interfaces to different instances.

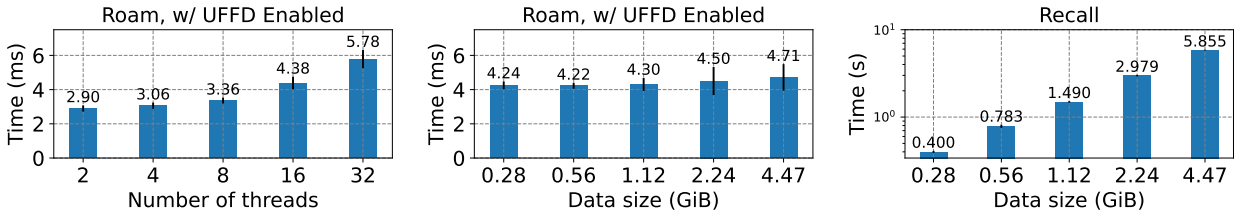


Fig. 3: Micro-benchmark of DIFFUSED's roam and recall operations (with UFFD enabled). Results show the mean roam time and standard deviation over 10 experiments. *(left)* Mean roam time for different numbers of threads. *(mid)* Mean roam time for various allocated memory sizes with fixed 16 threads, where UFFD is enabled and achieves extremely fast roam. *(right)* Mean recall time for various allocated memory sizes with fixed 16 threads, UFFD is disabled here since sending back memory contents within the revocation window is the first priority.

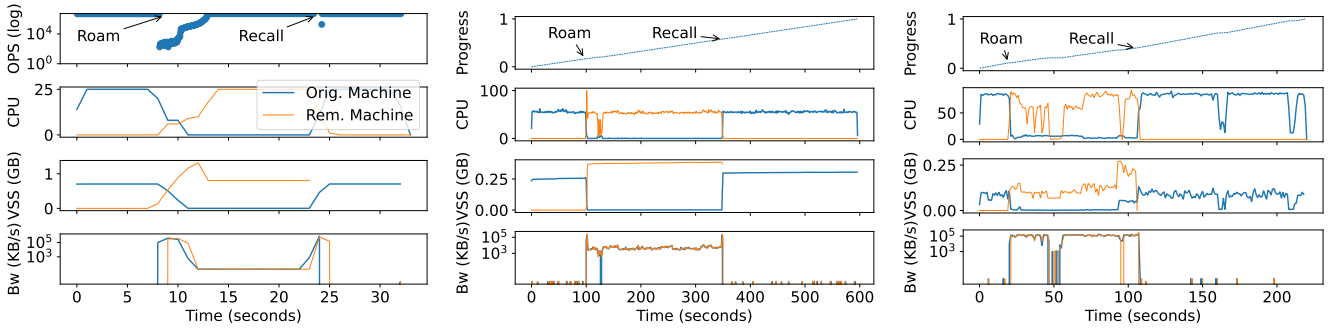


Fig. 4: Performance and resource utilization of roaming a program (single round). *(left)* Roaming SimpleKV with four threads and 10M records. *(mid)* Roaming an FFmpeg video conversion task. *(right)* Roaming a Lbzip2 compression task.

specified. We enable unprivileged userfaultfd and disable ASLR. For compatibility across CPU models, all cloud-hosted instances start with the `noxsave` kernel parameter unless otherwise specified. We use a separate instance in the same zone to run network benchmarks.

A. *Understanding* DIFFUSED *internals*

Experimental setup. We first evaluate DIFFUSED on our bare-metal cluster to avoid any potential influence from hypervisors [76, 77]. We use several servers with Intel Xeon E5-2620 v4 CPUs and 32 GB of memory and Ubuntu 22.04 (5.15.0-78-generic). DHOST and target programs run within an Alpine chroot v3.19 with patched musl-libc-1.2.4.

Micro-benchmarks, roam and recall overhead. We first explore how the number of threads affects roam time. Upon roaming, DHOST collects and marshals information of each thread. Figure 3 (*left*) shows the roam time when `SimpleKV` is loaded with 10M records. Roam time grows linearly with a small number of threads ($R^2 = .9946$). With more threads (e.g., 64), context switching incurs additional overhead.

Figure 3 (*mid*) shows the roam time as a function of the memory used by `SimpleKV`. Roam time is not significantly affected by allocated memory size with UFFD enabled (subsection IV-C) since only the memory layout is transferred. Figure 3 (*right*) shows the recall time, using the same configuration as Figure 3 (*mid*). Since DIFFUSED sends all allocated memory, the recall time scales linearly as the allocated memory increases. For networked programs, an additional 0.5-1s is required for enumerating network-related fds and events. Due to space constraints, we omit the roam/recall time with UFFD disabled, which scales linearly with allocated memory size (and is thus similar to Figure 3(*right*)).

Takeaways: The allocated memory size dominates recall time, which is crucial for revocation handling.

Performance and resource utilization. Given identical machines, a roamed program should ideally perform as well as its non-roamed counterpart. To verify this, we run `SimpleKV`, `FFmpeg`, and `Lbzip2` by roaming them (with UFFD enabled) until a steady state is reached, and then recall them. We record the overall CPU and network utilization of both machines and the physical memory utilization (VSS) of the DHOST processes from both sides (see Figure 4).

We start `SimpleKV` with four threads and 10 M records (totaling 0.56 GiB), as shown in Figure 4 *(left)*. DIFFUSED allows `SimpleKV` to operate nearly continuously, with a two-stage slowdown after roaming since `SimpleKV` triggers UFFD page synchronization. After DHOST sends all pages to the remote page cache, missing pages are installed more quickly. As pages are populated on the remote host, performance restores to that before the roam. `FFmpeg` and `Lbzip2` use less memory, yielding less noticeable roaming and recall times.

The original side's CPU and memory utilization drop when the program roams. The network is continuously used to access the original side's `stdio` file descriptors. `Lbzip2` triggers

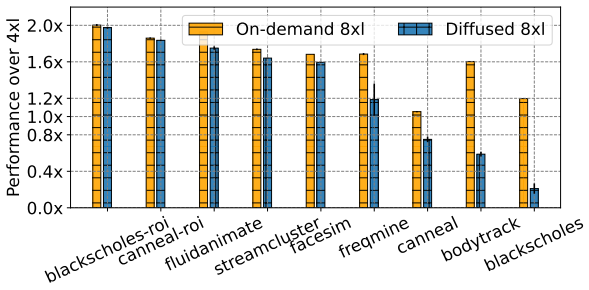


Fig. 5: PARSEC 3.0 performance of 32 worker threads and native (largest) workloads. Baseline: benchmarking with `c6in.4xlarge` (16 vCPUs) on-demand instance. On-demand 8xl: benchmarking with a `c6in.8xlarge` (32 vCPUs) on-demand instance compared to baseline. DIFFUSED 8xl: roamed performance from a `c6in.large` (2 vCPUs) on-demand instance to a `c6in.8xlarge` (32 vCPUs) spot instance, compared with baseline. 1.0x means the same performance as the baseline; higher is better.

more disk I/O via routed system calls and generates higher network utilization (100 MBps) than `FFmpeg` (1 MBps).

Takeaways: A roamed program mainly uses remote resources, and the network for relaying some system calls.

B. Scalability study: CPU-bound programs

Programs spawning multiple threads can scale when roamed to spot instances with more vCPUs.

Benchmarking PARSEC 3.0 suite. We first benchmark PARSEC 3.0 (native scale workloads) with a `c6in.4xl` (16 vCPU, $0.9072/hr) on-demand instance and repeat the same tests with a `c6in.8xl` (32 vCPU, $1.8144/hr) on-demand instance to get the baseline and on-demand 8xl improvements (`xl` is the abbr. of `xlarge`). We repeat the same but roamed benchmarks with an *always-cheaper* DIFFUSED deployment ($0.8391/hr) of a `c6in.large` (2vCPU) on-demand instance and a `c6in.8xlarge` (32vCPU) spot instance, and get the DIFFUSED 8xl improvements, as shown in Figure 5.

We observe multiple improvements of DIFFUSED except `canneal`, `bodytrack`, and `blackscholes`, which are affected mainly by forwarded disk IO. `Blackscholes` triggers 800k+ small disk system calls (*read(), writev()*) totaling 844 MB of data. `Canneal` and `bodytrack` also have similar behaviors. The ROI performance[4] of `blackscholes` and `canneal` show high scalability.

Takeaways: Multi-threaded programs without intensive small disk I/Os scale when roaming to larger instances.

Benchmarking disk-IO throughput. To understand the throughput of forwarded disk-IO system calls, we duplicate a 1.3GB file using `dd (busybox)` from a `i4i.large` instance with both default gp2 EBS volume and an attached

[4]Region Of Interest performance (i.e., the computation-only execution time of `blackscholes`, excluding the data preparation time).

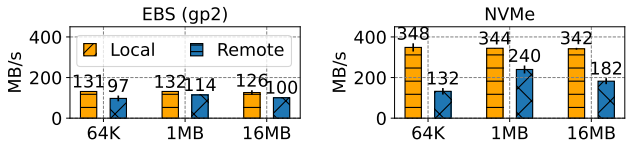


Fig. 6: Disk IO throughput with dd (busybox), duplicating a 1.3 GB file with an EBS and a `i4i` NVMe drive, respectively, with different bs values (e.g., bs=64K, 1MB, etc.). *Local*: running dd without DIFFUSED. *Remote*: roaming to a `c6i.2xlarge` instance and Disk-IOs are forwarded back.

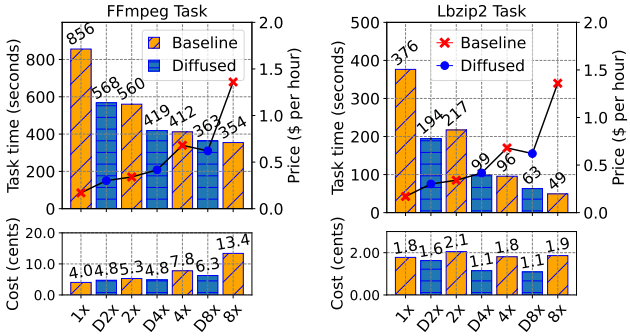
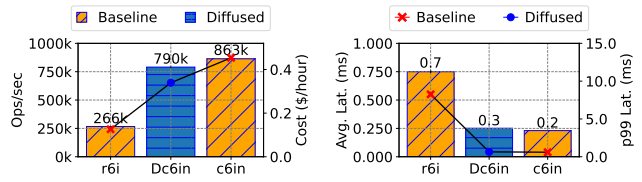


Fig. 7: Task finish times (upper bars) and time-based instance provisioning prices (lines) and per task costs (lower bars) of running `FFmpeg` video conversion and `Lbzip2` compression, (i) directly inside `c6i` on-demand instances of multiple scales (e.g., 2x for `c6i.2xlarge`) and roaming from a `i4i.large` on-demand instance to corresponding sizes of spot instances (e.g., `D4x` means roam to a `c6i.4xl` spot instance). Lower is better.

NVMe drive, as shown in Figure 6 "Local" columns. We then roam `dd` to another `c6i.2xlarge` instance to incur forwarded disk IOs ("Remote" columns in Figure 6), which are generally slower than local ones; forwarded disk IOs using NVMe can have similar or better throughput than local gp2 IOs.

Benchmarking FFmpeg and Lbzip2. We explore DIFFUSED's scalability and costs with `FFmpeg` and `Lbzip2`. First, we run `FFmpeg` video conversion task and `Lbzip2` compression task with multiple `c6i` on-demand instances (with gp2 EBS) of different scales. We next start with an `i4i.large` instance (with NVMe) and roam to different `c6i` instances.

The task finish times, instance prices, and cost per task are shown in Figure 7. The general trend shows that running such tasks under a DIFFUSED deployment (e.g., `D4x`) can achieve similar performance approaching its corresponding on-demand deployment (e.g., `4x`) while benefiting from the spot discount. Note that roaming the `Lbzip2` task to `c6i.2xlarge` spot instance performs better than running with a `2xlarge` instance natively since accessing remote NVMe storage is faster than accessing local gp2 EBS for this IO rate.



(a) Throughput (bar) and per hour cost (line).

(b) Mean latency (bar) and p99 latency (line).

Fig. 8: `Memcached` with `r6i.large` (2 vCPU, 16 GiB memory) and `c6in.2xlarge` (8 vCPU, 16 GiB memory) instances, loaded with 20 M 640 bytes records totalling 14 GiB. Baseline: running without DIFFUSED on `r6i` and `c6in` instances. DIFFUSED: roaming from `r6i` instance to the `c6in` instance.

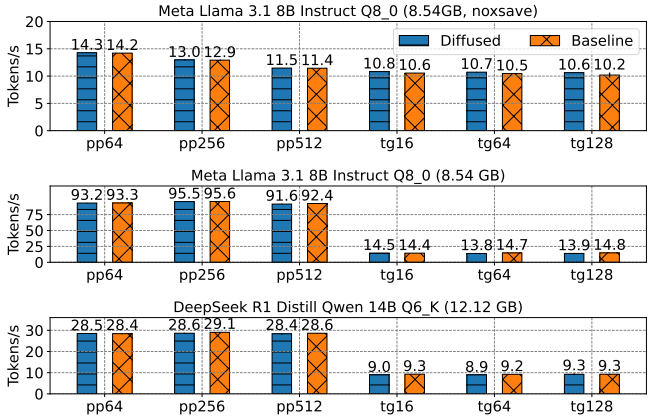


Fig. 9: `Llama-bench` results against Llama 3.1 8B quant 8bit and DeepSeek-R1 distill Qwen 14B quant 6bit models. Baseline: running `llama-bench` directly with `c6i.8xlarge` (32 vCPU, 64 GiB memory) instance. DIFFUSED: roaming `llama-bench` from a `r6i.xlarge` (4 vCPU, 32 GiB memory) instance. *(pp)* Prompt processing. *(tg)* Text generation.

Takeaways: DIFFUSED enables a flexible combination of on-demand and spot instances to stay both performant and cost-efficient.

C. Scalability study: Memory-bound programs.

To recall a memory-bound program safely, the on-demand instance should be a memory-optimized (e.g., `r6i`) instance with large enough memory to contain the roamed program.

Benchmarking Memcached. We benchmark `Memcached` with a memory-optimized `r6i.large` (2 vCPU, 16 GiB) and a compute-optimized `c6in.2xlarge` (8 vCPU, 16 GiB) instances. We first run `Memcached` with two and eight threads with these two instances, respectively, loaded with 14 GiB data and provide the throughput and latency results as the baseline results. We next roam a loaded `Memcached` process (eight threads) from the `r6i` instance to the `c6in` instance and get DIFFUSED results. As shown in Figure 8, a roamed `Memcached` can scale well while saving the instance provision costs. Additionally, roam to `c6in.2xlarge` takes 13.13s ($\pm 0.0023s$) and recall to `r6i.large` takes 12.90s ($\pm 0.0008s$).

Benchmarking Llama.cpp. We benchmark `Llama.cpp` with its builtin `llama-bench`, first on a `c6i.8xlarge` instance and then roam to the `c6i.8xlarge` from a `r6i.xlarge` instance. Since both instances use the same CPUs, we disable the `noxsave` kernel parameter for a higher prompt processing token rate (e.g., 14.3 to 93.2 for pp64, Llama model). A roamed `llama-bench` can yield similar token generate rates compared to running it natively with a larger instance (Figure 9).

Revocation for `Llama.cpp` may be unnecessary since the deployed model stays unchanged, and the input/output is small. Roaming `Llama.cpp` is faster than loading the model from EBS snapshot that triggers EBS initialization [44] but is similar to loading the model from a detachable EBS.

D. Diffusing multiple programs

DIFFUSED can balance multiple applicable processes across instances for better resource utilization when available.

When original instance overloads. Our results (Figure 10) show that when roaming to a more powerful instance, `FFmpeg`, `Lbzip2`, and `Memcached` benefit from a performance boost, even when overloading the original instance with `stress-ng` (8 CPU stressors). Single-thread `Tornado` can still benefit from a remote idle core when the original instance overloads.

Diffusing multiple programs in real-time. We then scale out multiple programs toward a group of spot instances with a DMON policy to roam all tasks as long as there are idle DHOST runtimes. We keep a `c6in.xlarge` original instance and spawn a group of `c7i.4xlarge` spot instances.

We start 3 `FFmpeg` and 2 `Lbzip2` tasks (named *FT 1-3* and *LT 1-2*) from the original instance, and these tasks overload each other (FTs report an 11.8x ETA, and LTs report a 2.6x ETA compared with individual trials). We then spawn four spot instances that automatically fit three FTs and one LT, and the remaining LT resumes its performance. We then use AWS FIS to terminate a spot instance and force FT1 to be recalled, and FT1 runs slower since it must share CPU resources with LT2. We finally spawn another two spot instances for FT1 and LT2, and all tasks run faster.

Takeaways: DMON scales toward available spot instances, and tenants need not be cognizant of revocations.

E. Other migration based approaches

We compare DIFFUSED with several state-of-the-art migration approaches as shown in Table II.

Process migration (CRIU [78]). For revocation handling with CRIU, lazy migration and incremental checkpointing are orthogonal; the former is less critical to store checkpointed images safely, and the latter does not help if being revoked before a checkpoint is ready. For EC2, non-memory-bound programs can be directly checkpointed to (i) a detachable gp2

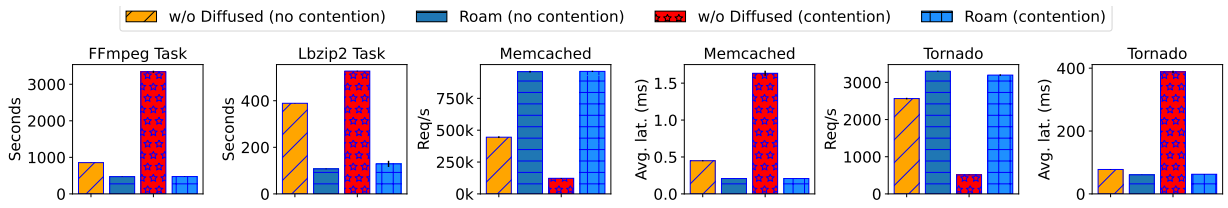


Fig. 10: Diffusing programs between EC2 instances while overloading the original instance with a stress-ng CPU stressor.

TABLE II: Comparison with CRIU and MicroVM (Firecracker).

Program	Approach (Platform)	Configuration	Elapsed Time (s) Checkpoint	Restore	Reattach gp2	Total time (s)
`FFmpeg`	CRIU (EC2)	detachable gp2	0.194 (±0.006)	0.981 (±0.004)	7.35 (±1.02)	8.525 (10.65× longer)
		NFS (NVMe-backed)	0.58 (±0.012)	0.55 (±0.04)	-	1.13 (1.41× longer)
	DIFFUSED (EC2)	UFFD disabled	0.80 (±0.004), DIFFUSED recall			0.80 (baseline)
`Memcached` (7GiB)	CRIU (EC2)	Page server (tmpfs)	12.52 (±0.02)	2.77 (±0.02)	-	15.29 (1.23× longer)
	DIFFUSED (EC2)	UFFD disabled	12.39 (±0.004), DIFFUSED recall			12.39 (baseline)
`Memcached` (7GiB)	Firecracker (GCP)	NFS (tmpfs-backed)	7.95 (±0.08)	0.03 (±0.002)	-	7.98 (1.25× longer)
	DIFFUSED (GCP)	UFFD disabled	6.37 (±0.048), DIFFUSED recall			6.37 (baseline)

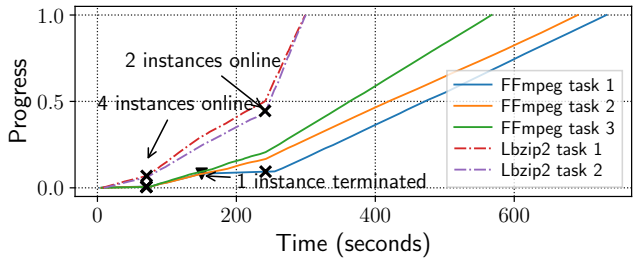


Fig. 11: Scaling out with more spot instances. ✖: Roaming programs. ▼: Recall programs upon revocation.

EBS volume or (ii) an NFS-mounted remote volume (e.g., NVMe from on-demand `i4i` instances). DIFFUSED recalls the `FFmpeg` task in 0.80s ($\pm 0.004s$) with UFFD disabled and is faster than CRIU (1.41× to 10.65×) equipped with detachable EBS or remote NFS. For `Memcached`, a DIFFUSED recall only takes $12.39s(\pm 0.004s)$ compared to slower CRIU (1.23×) with a remote tmpfs-backed page server [79].

MicroVM migration (Firecracker [80]). Running Firecracker inside spot instances requires nested virtualization, which Amazon EC2 disables but Google Cloud Platform (GCP) allows. Considering GCP's 30s revocation window, we use a memory-backed NFS drive, and checkpointing/restoring a `Memcached` MicroVM with 7 GiB data takes 1.25× as long compared to 6.37s for DIFFUSED to perform a recall.

Full VM migration. Existing VM-level migration handling revocation [7, 8] also requires nested virtualization. We run live migration with GCP-nested KVM instances using libvirt, and migrating a four vCPU nested VM with 4GiB memory and 8 GB storage (inside an `N1-standard-4` instance) can take up to 50s, exceeding the GCP revocation limit (30s).

Takeaways: Migration requires tenants to ensure storage consistency. DIFFUSED recalls faster by comparison.

VI. DISCUSSION AND LIMITATIONS

Comparison with serverless. AWS Lambda [25] poses constraints with per-function limits (e.g., 900s execution, 10 GiB memory usage). Spot instances have no such limitations, and DIFFUSED automatically handles revocation.

Comparison with VM techniques. Modern clouds provide automated instance provisioning, or *autoscaling*, for scalability [36].DIFFUSED complements this by offloading workloads to spot instances within the group and automatically recalling execution when instances are revoked or scaled down. *Burstable VMs* can temporarily use more than assigned resources to handle potential load spikes [81] but can only use resources of the same physical machine, whereas DIFFUSED allows programs to use resources from additional physical machines without having to manage burstable resources [82]. *Harvested VMs* can "harvest" idle CPU or memory resources and must return harvested resources once required by the physical machine [83]; we envision that tenants could also deploy DIFFUSED to harvested VMs.

Best-effort vs. guaranteed safe revocation handling. This paper provides a best-effort estimate of the maximum amount of memory the program can recall safely within the revocation window without the cloud's cooperation. However, the cloud can provide DIFFUSED-enabled spot instances that explicitly revoke with DIFFUSED's recall routine rather than blindly setting a pre-configured revocation window (e.g., 30s) to guarantee a safe revocation. Additionally, the cloud may temporarily increase the bandwidth of DIFFUSED's memory channel for faster memory state transfer.

Providing transparency and seamless experience. Recalling a program with a large memory footprint can cause it to halt for up to the revocation window length, harming

the seamless experience. However, once the cloud provides DIFFUSED-enabled instances, UFFD can be enabled for the recall procedure, thereby substantially reducing the halt time. Current DIFFUSED implementation closes network connections upon roam and recall. We envision integrating with a user-space network layer (e.g., AF_XDP-based iip [59]) could enable roam/recall with existing connections.

Cloud provider support. We use AWS EC2 as a representative case study, but DIFFUSED generalizes to other cloud providers. To support a cloud provider, OS images need to be created to autostart DHOST and connect to DMON. Additionally, DMON needs to connect to the cloud's lifecycle API to receive termination notifications. Microsoft Azure [84] and GCP [85] support spot (preemptive) instances. Similar to AWS EC2, GCP provides Preemption Signals.

Program support. DIFFUSED supports multi-threaded POSIX programs for Linux. Currently, DIFFUSED does not support multi-process programs; adding support requires managing the entire process group and IPC objects. DIFFUSED does not support programs that implicitly write to memory-mapped files or devices since the remote side memory is not backed by corresponding files or devices. DIFFUSED does not currently support manually managed threads (i.e., threads not created using pthread_create), manually managed memory (e.g., usage of MMAP_FIXED that uses hardcoded addresses), or assembly-level system calls.

Syscall support. Most programs use few system calls [86]. Supporting all system calls is left as a future engineering task, requiring the insertion of a lightweight shim rather than a complete reimplementation; DIFFUSED already supports 50+ system calls, which is sufficient for a swath of programs.

Data integrity and state preservation. DIFFUSED's roam and recall operations only move (not modify) CPU contexts and memory content and potentially handle sockets for networked programs. To preserve integrity, DIFFUSED employs binary and memory isolation to avoid tampering with program code and data.

VII. CONCLUSION

We introduce DIFFUSED, an implementation of *process diffusion* that turns spot instances into diskless computation nodes alongside on-demand instances in order to free tenants from handling spot instance revocation. DIFFUSED balances availability, performance, and cost, and re-enables existing traditional programs to benefit from spot instances without worrying about revocation handling.

ACKNOWLEDGEMENTS

We thank the anonymous reviewers for their valuable comments and suggestions. This material is supported in part by the Callahan Chair Fund and the Farr Faculty Award. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of Georgetown University.

REFERENCES

[1] AWS, "Spot Instances - AWS EC2," 2024, Available at https://aws.amazon.com/ec2/spot.

[2] Alibaba, "What are preemptible instances?" 2024, Available at https://www.alibabacloud.com/help/en/ecs/user-guide/overview-4.

[3] Amazon Web Services, Inc. or its affiliates., "AWS Pricing Calculator," 2024, Available at https://calculator.aws.

[4] ——, "Spot Instance interruption notices," 2024, Available at https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-instance-termination-notices.html.

[5] C. Delimitrou and C. Kozyrakis, "HCloud: Resource-efficient provisioning in shared cloud systems," *SIGARCH Comput. Archit. News*, vol. 44, no. 2, p. 473–488, Mar. 2016. [Online]. Available: https://doi.org/10.1145/2980024.2872365

[6] P. Sharma, S. Lee, T. Guo, D. Irwin, and P. Shenoy, "SpotCheck: designing a derivative IaaS cloud on the spot market," in *Proceedings of the Tenth European Conference on Computer Systems*, ser. EuroSys '15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: https://doi.org/10.1145/2741948.2741953

[7] X. He, P. Shenoy, R. Sitaraman, and D. Irwin, "Cutting the cost of hosting online services using cloud spot markets," in *Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 207–218. [Online]. Available: https://doi.org/10.1145/2749246.2749275

[8] S. Subramanya, T. Guo, P. Sharma, D. Irwin, and P. Shenoy, "SpotOn: a batch computing service for the spot market," in *Proceedings of the Sixth ACM Symposium on Cloud Computing*, ser. SoCC '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 329–341. [Online]. Available: https://doi.org/10.1145/2806777.2806851

[9] T. Benjaponpitak, M. Karakate, and K. Sripanidkulchai, "Enabling live migration of containerized applications across clouds," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 2529–2538.

[10] criu.org, "The CRIU Project," 2024, Available at https://criu.org/Main_Page.

[11] J. Ansel, K. Arya, and G. Cooperman, "DMTCP: Transparent checkpointing for cluster computations and the desktop," in *2009 IEEE international symposium on parallel & distributed processing*. IEEE, 2009, pp. 1–12.

[12] Z. Wu, W.-L. Chiang, Z. Mao, Z. Yang, E. Friedman, S. Shenker, and I. Stoica, "Can't be late: Optimizing spot instance savings under deadlines," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. Santa Clara, CA: USENIX Association, Apr. 2024, pp. 185–203. [Online]. Available: https://www.usenix.org/conference/nsdi24/presentation/wu-zhanghao

[13] C. Zhang, M. Yu, W. Wang, and F. Yan, "MArk: Exploiting cloud services for Cost-Effective, SLO-Aware machine learning inference serving," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. Renton, WA: USENIX Association, Jul. 2019, pp. 1049–1062. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/zhang-chengliang

[14] C. Wang, B. Urgaonkar, A. Gupta, G. Kesidis, and Q. Liang, "Exploiting spot and burstable instances for improving the cost-efficacy of in-memory caches on the public cloud," in *Proceedings of the Twelfth European Conference on Computer Systems*, ser. EuroSys '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 620–634. [Online]. Available: https://doi.org/10.1145/3064176.3064220

[15] Z. Xu, C. Stewart, N. Deng, and X. Wang, "Blending on-demand and spot instances to lower costs for in-memory storage," in *IEEE INFOCOM 2016 - The 35th Annual IEEE International Conference on Computer Communications*, 2016, pp. 1–9.

[16] C. Qu, R. N. Calheiros, and R. Buyya, "A reliable and cost-efficient auto-scaling system for web applications using heterogeneous spot instances," *J. Netw. Comput. Appl.*, vol. 65, no. C, p. 167–180, apr 2016. [Online]. Available: https://doi.org/10.1016/j.jnca.2016.03.001

[17] AWS, "Prepare for Spot Instance interruptions," 2024, Available at https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/prepare-for-interruptions.html.

[18] F. Yang, L. Wang, Z. Xu, J. Zhang, L. Li, B. Qiao, C. Couturier, C. Bansal, S. Ram, S. Qin, Z. Ma, I. n. Goiri, E. Cortez, T. Yang, V. Rühle, S. Rajmohan, Q. Lin, and D. Zhang, "Snape: Reliable and low-cost computing with mixture of spot and on-demand VMs," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS 2023. New York, NY, USA:

Association for Computing Machinery, 2023, p. 631–643. [Online]. Available: https://doi.org/10.1145/3582016.3582028

[19] S. Yang, S. Khuller, S. Choudhary, S. Mitra, and K. Mahadik, "Scheduling ML training on unreliable spot instances," in *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing Companion*, ser. UCC '21. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: https://doi.org/10.1145/3492323.3495594

[20] Y. Gong, B. He, and A. C. Zhou, "Monetary cost optimizations for MPI-based HPC applications on amazon clouds: checkpoints and replicated execution," in *SC '15: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015, pp. 1–12.

[21] L. Zhang, J. Litton, F. Cangialosi, T. Benson, D. Levin, and A. Mislove, "Picocenter: supporting long-lived, mostly-idle applications in cloud environments," in *Proceedings of the Eleventh European Conference on Computer Systems*, ser. EuroSys '16. New York, NY, USA: Association for Computing Machinery, 2016. [Online]. Available: https://doi.org/10.1145/2901318.2901345

[22] O. Laadan and S. E. Hallyn, "Linux-CR: Transparent application checkpoint-restart in Linux," in *Linux Symposium*, vol. 159. Citeseer, 2010.

[23] The CRIU Project, "What Can Change After C/R," 2023, Available at https://criu.org/What_can_change_after_C/R.

[24] ——, "What Cannot be Checkpointed," 2023, Available at https://criu.org/What_cannot_be_checkpointed.

[25] AWS, "AWS Lambda - Serverless Compute," 2024, Available at https://aws.amazon.com/lambda/.

[26] Microsoft, "Azure Functions Serverless Compute," 2024, Available at https://azure.microsoft.com/en-us/services/functions/.

[27] ——, "Azure App Service," 2024, Available at https://azure.microsoft.com/en-us/services/app-service/.

[28] Z. Ruan, S. J. Park, M. K. Aguilera, A. Belay, and M. Schwarzkopf, "Nu: Achieving Microsecond-Scale resource fungibility with logical processes," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 1409–1427. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/ruan

[29] S.-H. Kim, H.-R. Chuang, R. Lyerly, P. Olivier, C. Min, and B. Ravindran, "DEX: scaling applications beyond machine boundaries," in *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2020, pp. 864–876.

[30] A. Barbalace, M. Sadini, S. Ansary, C. Jelesnianski, A. Ravichandran, C. Kendir, A. Murray, and B. Ravindran, "Popcorn: bridging the programmability gap in heterogeneous-ISA platforms," in *Proceedings of the Tenth European Conference on Computer Systems*, ser. EuroSys '15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: https://doi.org/10.1145/2741948.2741962

[31] M. S. Gordon, D. A. Jamshidi, S. Mahlke, Z. M. Mao, and X. Chen, "COMET: Code offload by migrating execution transparently," in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. Hollywood, CA: USENIX Association, Oct. 2012, pp. 93–106. [Online]. Available: https://www.usenix.org/conference/osdi12/technical-sessions/presentation/gordon

[32] Google, "Google App Engine," 2024, Available at https://cloud.google.com/appengine/.

[33] AWS, "Amazon Compute Service Level Agreement," 2022, Available at https://aws.amazon.com/compute/sla/.

[34] O. Hadary, L. Marshall, I. Menache, A. Pan, E. E. Greeff, D. Dion, S. Dorminey, S. Joshi, Y. Chen, M. Russinovich, and T. Moscibroda, "Protean: VM allocation service at scale," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 845–861. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/hadary

[35] Amazon Web Services, Inc. or its affiliates., "Amazon EC2 On-Demand Pricing," 2024, Available at https://aws.amazon.com/ec2/pricing/on-demand/#Data_Transfer_within_the_same_AWS_Region.

[36] AWS, "AWS Auto Scaling," 2024, Available at https://aws.amazon.com/autoscaling/.

[37] V. A. Sartakov, L. Vilanova, and P. Pietzuch, "CubicleOS: A library OS with software componentisation for practical isolation," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 546–558.

[38] Z. Shen, Z. Sun, G.-E. Sela, E. Bagdasaryan, C. Delimitrou, R. Van Renesse, and H. Weatherspoon, "X-containers: Breaking down barriers to improve performance and isolation of cloud-native containers," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 121–135.

[39] A. Madhavapeddy, T. Leonard, M. Skjegstad, T. Gazagnaire, D. Sheets, D. Scott, R. Mortier, A. Chaudhry, B. Singh, J. Ludlam *et al.*, "Jitsu: Just-In-Time summoning of unikernels," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, 2015, pp. 559–573.

[40] C.-C. Tsai, D. E. Porter, and M. Vij, "Graphene-SGX: A practical library OS for unmodified applications on SGX," in *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, 2017, pp. 645–658.

[41] musl-libc.org, "The Musl-libc Library," 2024, Available at https://www.musl-libc.org/.

[42] Alpine Linux Development Team, "Index — Alpine Linux," 2024, Available at https://www.alpinelinux.org/.

[43] J. T. Lim and J. Nieh, "Optimizing nested virtualization performance using direct virtual hardware," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 557–574. [Online]. Available: https://doi.org/10.1145/3373376.3378467

[44] AWS, "Initialize Amazon EBS volumes," 2024, Available at https://docs.aws.amazon.com/ebs/latest/userguide/ebs-initialize.html.

[45] R. Bianchini, L. I. Kontothanassis, R. Pinto, M. De Maria, M. Abud, and C. L. d. Amorim, "Hiding communication latency and coherence overhead in software DSMs," *ACM SIGOPS Operating Systems Review*, vol. 30, no. 5, pp. 198–209, 1996.

[46] A. Itzkovitz and A. Schuster, "Multiview and millipage-fine-grain sharing in page-based DSMs," in *OSDI*, vol. 99, 1999, pp. 215–228.

[47] M. K. Aguilera, N. Amit, I. Calciu, X. Deguillard, J. Gandhi, P. Subrahmanyam, L. Suresh, K. Tati, R. Venkatasubramanian, and M. Wei, "Remote memory in the age of fast networks," in *Proceedings of the 2017 Symposium on Cloud Computing*, ser. SoCC '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 121–127. [Online]. Available: https://doi.org/10.1145/3127479.3131612

[48] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal *et al.*, "Pond: CXL-based memory pooling systems for cloud platforms," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 574–587.

[49] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, and P. Chauhan, "TPP: Transparent page placement for CXL-enabled tiered-memory," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023, pp. 742–755.

[50] C. Guo, H. Wu, Z. Deng, G. Soni, J. Ye, J. Padhye, and M. Lipshteyn, "RDMA over commodity ethernet at scale," in *Proceedings of the 2016 ACM SIGCOMM Conference*, 2016, pp. 202–215.

[51] "mmap(2) — Linux manual page," 2024, Available at https://man7.org/linux/man-pages/man2/mmap.2.html.

[52] Linux man-pages, "Userfaultfd(2) — Linux manual page," 2024, Available at https://man7.org/linux/man-pages/man2/userfaultfd.2.html.

[53] criu.org, "Page server - CRIU," 2024, Available at https://criu.org/Page_server.

[54] J. K. Ousterhout, A. R. Cherenson, F. Douglis, M. N. Nelson, and B. B. Welch, "The Sprite network operating system," *Computer*, vol. 21, no. 2, pp. 23–36, 1988.

[55] A. Barak and O. La'adan, "The MOSIX multicomputer operating system for high performance cluster computing," *Future Generation Computer Systems*, vol. 13, no. 4-5, pp. 361–372, 1998.

[56] MOSIX authors, "MOSIX FAQ: Can I Run Threaded Applications," 2024, Available at https://mosix.cs.huji.ac.il/faq/output/faq_q042.html.

[57] "Futex(7) — Linux Manual Page," 2024, Available at https://man7.org/linux/man-pages/man7/futex.7.html.

[58] AWS, "Elastic network interfaces," 2024, Available at https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-eni.html.

[59] K. Yasukata, "iip: An integratable TCP/IP stack," *SIGCOMM Comput. Commun. Rev.*, vol. 54, no. 1, p. 21–28, Aug. 2024. [Online]. Available: https://doi.org/10.1145/3687230.3687233

[60] I. Choi, N. Wadekar, R. Joshi, J. Fried, D. R. K. Ports, I. Zhang, and J. Li, "Capybara: μSecond-Scale Live TCP Migration," in *Proceedings*

of the 14th ACM SIGOPS Asia-Pacific Workshop on Systems, ser. APSys '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 30–36. [Online]. Available: https://doi.org/10.1145/3609510.3609813

[61] J. Nagle, "Congestion Control in IP/TCP Internetworks," *ACM SIGCOMM Computer Communication Review*, vol. 14, no. 4, pp. 11–17, 1984.

[62] BusyBox authors, "BusyBox," 2024, Available at https://busybox.net/.

[63] ffmpeg.org, "FFmpeg - A complete, cross-platform solution to record, convert and stream audio and video," 2024, Available at https://ffmpeg.org/.

[64] lbzip2.org, "LBZIP2 - parallel bzip2 compression utility," 2024, Available at https://lbzip2.org/.

[65] ggml-org, "LLM inference in C/C++," 2025, Available at https://github.com/ggml-org/llama.cpp.

[66] Bartowski, "Meta-Llama-3.1-8B-Instruct-GGUF ," 2025, Available at https://huggingface.co/bartowski/Meta-Llama-3.1-8B-Instruct-GGUF/blob/main/Meta-Llama-3.1-8B-Instruct-Q8_0.gguf.

[67] ——, "DeepSeek-R1-Distill-Qwen-14B-GGUF," 2025, Available at https://huggingface.co/bartowski/DeepSeek-R1-Distill-Qwen-14B-GGUF/blob/main/DeepSeek-R1-Distill-Qwen-14B-Q6_K.gguf.

[68] The Memcached Authors, "Memcached, a distributed memory object caching system," 2024, Available at https://memcached.org/.

[69] Redis Labs, "Memtier_benchmark, a NoSQL Redis and Memcache traffic generation and benchmarking tool," 2024, Available at https://github.com/RedisLabs/memtier_benchmark.

[70] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The PARSEC benchmark suite: characterization and architectural implications," in *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT '08. New York, NY, USA: Association for Computing Machinery, 2008, p. 72–81. [Online]. Available: https://doi.org/10.1145/1454115.1454128

[71] Chameleon, "Chameleon Resource Browser," 2024, Available at https://www.chameleoncloud.org/hardware/.

[72] The Tornado Authors, "Tornado web server," 2024, Available at https://www.tornadoweb.org/en/stable/.

[73] Will Glozer, "Wrk - A HTTP Benchmarking Tool," 2024, Available at https://github.com/wg/wrk.

[74] Colin Ian King, "stress-ng (stress next generation)," 2024, Available at https://github.com/ColinIanKing/stress-ng.

[75] AWS, "AWS Fault Injection Service," 2024, Available at https://aws.amazon.com/fis/.

[76] Y. Liu, T. Xu, Z. Mi, Z. Hua, B. Zang, and H. Chen, "CPS: A cooperative para-virtualized scheduling framework for manycore machines," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2023, pp. 43–56.

[77] S. Bergman, M. Silberstein, T. Shinagawa, P. Pietzuch, and L. Vilanova, "Translation Pass-Through for Near-Native paging performance in VMs," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 753–768. [Online]. Available: https://www.usenix.org/conference/atc23/presentation/bergman

[78] The CRIU Project, "Checkpoint/Restore In Userspace," 2023, Available at https://criu.org/Main_Page.

[79] criu.org, "Disk-less migration," 2024, Available at https://criu.org/Disk-less_migration.

[80] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Santa Clara, CA: USENIX Association, Feb. 2020, pp. 419–434. [Online]. Available: https://www.usenix.org/conference/nsdi20/presentation/agache

[81] AWS, "Burstable Performance Instances - Amazon Elastic Compute Cloud," 2024, Available at https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/burstable-performance-instances.html.

[82] A. Ali, R. Pinciroli, F. Yan, and E. Smirni, "It's not a sprint, it's a marathon: Stretching multi-resource burstable performance in public clouds (industry track)," in *Proceedings of the 20th International Middleware Conference Industrial Track*, ser. Middleware '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 36–42. [Online]. Available: https://doi.org/10.1145/3366626.3368130

[83] Y. Wang, K. Arya, M. Kogias, M. Vanga, A. Bhandari, N. J. Yadwadkar, S. Sen, S. Elnikety, C. Kozyrakis, and R. Bianchini, "SmartHarvest: Harvesting idle CPUs safely and efficiently in the cloud," in *Proceedings of the Sixteenth European Conference on Computer Systems*, 2021, pp. 1–16.

[84] Microsoft, "Use Azure Spot Virtual Machine," 2024, Available at https://learn.microsoft.com/en-us/azure/virtual-machines/spot-vms.

[85] Google, "Preemptible VM instances," 2024, Available at https://cloud.google.com/compute/docs/instances/preemptible.

[86] N. DeMarinis, K. Williams-King, D. Jin, R. Fonseca, and V. P. Kemerlis, "Sysfilter: Automated system call filtering for commodity software," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, 2020, pp. 459–474.